AI-ENABLED SMART ATTENDANCE SYSTEM

With Face Recognition & Geofencing

UCS503 - Software Engineering Project Report
End-Semester Evaluation

Submitted By:

Name	**Roll No**
Charu Goyal	102303113
Yuvraj Sharma	102303119
Nandini Puri	102303092
Paavni Sharma	102313052

BE Third Year, COE
Group No: 3C14

Submitted To:
Dr. Vinod Kumar Bhalla

THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY
(Deemed to be University)
Computer Science and Engineering Department
Patiala, Punjab

Contents

1 Project Selection Phase **1**
1.1 Software Bid . . . 1
1.2 Project Overview . . . 1
1.3 Definitions, Abbreviations and Acronyms . . . 2

2 Analysis Phase **3**
2.1 Use Cases . . . 3
2.1.1 Use-Case Diagram . . . 3
2.1.2 Use Case Templates . . . 5
2.2 Activity Diagram . . . 6
2.3 Data Flow Diagrams (DFD) . . . 8
2.3.1 DFD Level 0 (Context Diagram) . . . 8
2.3.2 DFD Level 1 . . . 9
2.3.3 DFD Level 2 . . . 10

3 Software Requirements Specification (SRS) **11**
3.1 Introduction . . . 12
3.2 Overall Description . . . 13
3.3 Specific Requirements . . . 16
3.4 System Models . . . 18
3.5 Change History . . . 18
3.6 Document Approvers . . . 19

4 Design Phase **20**
4.1 Class Diagram . . . 20
4.2 Sequence Diagram . . . 22
4.3 Collaboration Diagram . . . 23

5 Deployment Phase **25**
5.1 Deployment Diagram . . . 25
5.2 Showcase of Implemented Version . . . 26

List of Figures

1 Use Case Diagram 4
2 Activity Diagram (Attendance Logic) 7
3 DFD Level 0: Context Diagram 8
4 DFD Level 1: Detailed Data Flow 9
5 DFD Level 2: Component Detail 10
6 Class Diagram 20
7 Sequence Diagram 22
8 Collaboration Diagram 23
9 Deployment Diagram for AI-ENABLED SMART ATTENDANCE SYSTEM 25
10 Implemented Login Interface 26
11 Implemented Student Dashboard 26

1 Project Selection Phase

1.1 Software Bid

Software Bid/Project Teams
UCS503 - Software Engineering Lab
Group: 3C14 **Dated:** March 18, 2026
Team Name: Tech Visionaries

Team Members:

Name	**Roll No**	**Project Experience**	**Programming Language used**
Charu Goyal	102303113	E-Commerce Web App	JavaScript (React.js)
Yuvraj Sharma	102303119	Library Management Sys	Python
Nandini Puri	102303092	Real-time Chat App	Python (FastAPI)
Paavni Sharma	102313052	Weather Predictor	Python, MongoDB

Choices of Projects:

Choice	**Project Name**	**Unique Selling Point (USP)**
First	**AI-ENABLED SMART ATTENDANCE SYSTEM**	1. **Eliminate Proxy Attendance:** Uses facial embeddings for identity verification. 2. **Location Validation:** GPS geofencing ensures physical presence. 3. **Real-time Reporting:** Instant insights for teachers/admins. 4. **Secure Access:** Role-based authentication.
Second	Smart Traffic Management System	1. Real-time traffic density control using AI. 2. Emergency vehicle prioritization (Ambulance/Fire). 3. Automated violation detection via camera feeds. 4. Dynamic signal timing optimization based on flow.

1.2 Project Overview

About the Project:
The **AI-ENABLED SMART ATTENDANCE SYSTEM** is designed to eliminate

proxy attendance and automate the manual tracking process. It leverages **Face Recognition (FaceNet)** for identity verification and **Geofencing (GPS)** to ensure students are physically present in the classroom. The system supports three user roles: Students, Teachers, and Admins, providing a seamless interface for marking and monitoring attendance.

Core Objectives:

1. **Eliminate Proxy Attendance:** Use facial embeddings to verify student identity.
2. **Location Validation:** Ensure students are within the classroom boundary using GPS checks.
3. **Real-time Reporting:** Provide instant attendance insights for teachers and admins.
4. **Secure Access:** Role-based authentication (Student, Teacher, Admin).

Technology Stack:

- **Frontend:** React.js 18, Vanilla JavaScript, HTML/CSS
- **Backend:** FastAPI (Python), FaceNet (Transfer Learning)
- **Database:** MongoDB
- **Security:** GPS Geofencing

1.3 Definitions, Abbreviations and Acronyms

Term/Acronym	Definition/Description
AI	Artificial Intelligence; simulation of human intelligence processes by computer systems.
GPS	Global Positioning System; a satellite-based radionavigation system used for geolocation.
Geofencing	A location-based service that uses GPS to create a virtual geographic boundary, triggering a response when a mobile device enters or leaves a particular area.
FaceNet	A deep learning architecture used for face recognition, mapping face images to a Euclidean space where distances correspond to face similarity.
PWA	Progressive Web Application; software delivered through the web that offers an app-like experience on mobile devices.
API	Application Programming Interface; a set of protocols for building and integrating application software.
JWT	JSON Web Token; a standard for securely transmitting information between parties as a JSON object, used here for secure authentication.
SRS	Software Requirements Specification; a document that describes the features and behavior of the software system to be developed.

2 Analysis Phase

2.1 Use Cases

User Features (Student):

1. **Authentication:** Secure login and registration for all users.
2. **Face Enrollment:** Capture and store face embeddings for identity verification.
3. **Mark Attendance:** Scan face to mark presence within the designated geofence.
4. **View History:** Check subject-wise attendance logs and percentage.

User Features (Teacher):

1. **Authentication:** Secure login for faculty members.
2. **Session Management:** Schedule classes and define geofencing boundaries.
3. **Attendance Monitoring:** View and manage student attendance records.
4. **Manual Override:** Modify attendance status in case of technical issues.

Administrator Features:

1. **Authentication:** Secure login for admin access.
2. **User Management:** Add, update, or remove student and teacher profiles.
3. **System Maintenance:** Manage course lists and monitor system health.

2.1.1 Use-Case Diagram

The Use Case diagram below depicts the interactions between the Student, Teacher, and Admin actors. Key use cases include Face Enrollment, Mark Attendance (Face Scan), Start/End Session, and View Reports.

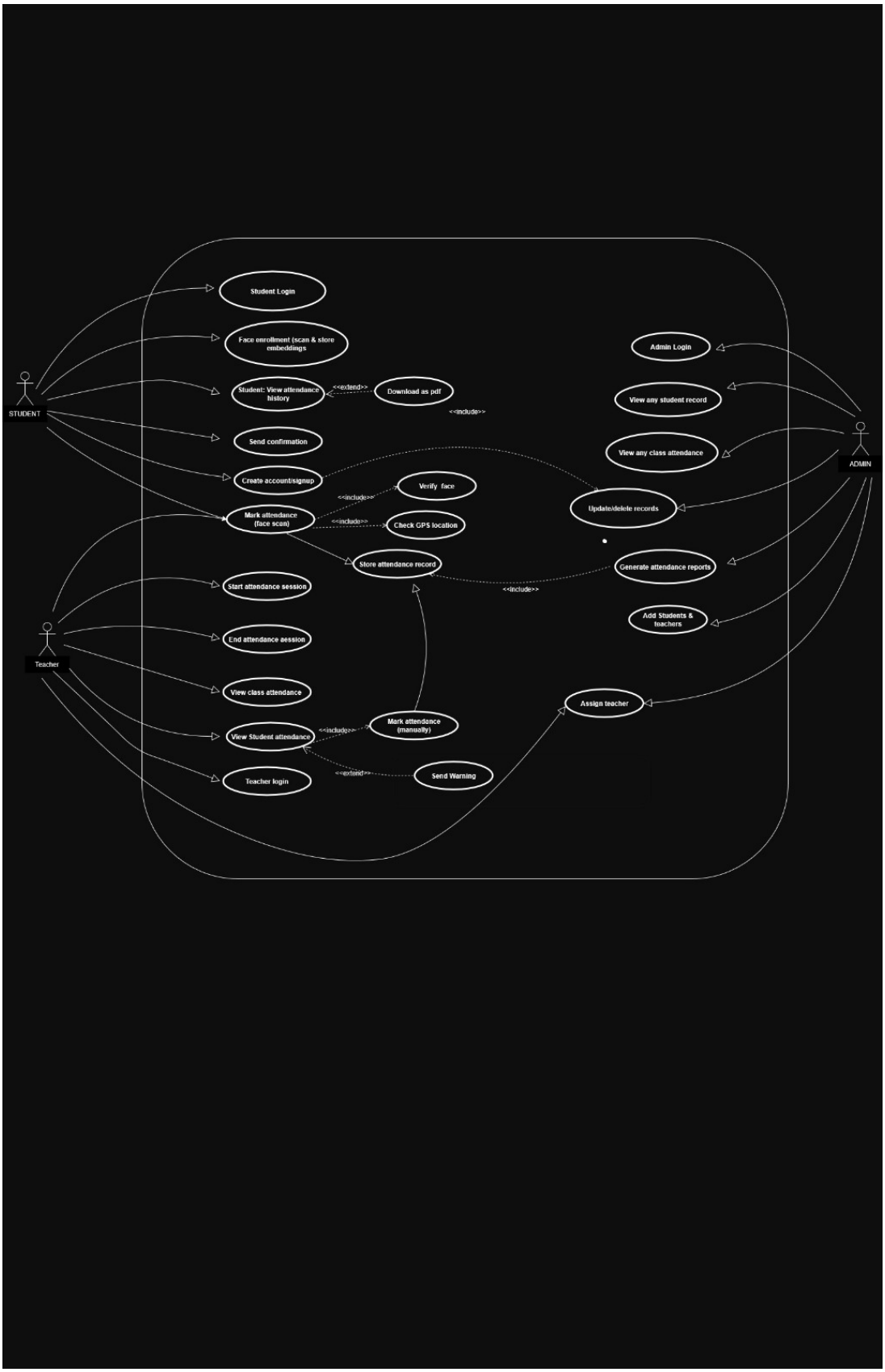

Figure 1: Use Case Diagram

2.1.2 Use Case Templates

Use Case Name	**Mark Attendance**
Actor	Student
Pre-condition	User logged in, within class time & location.
Description	Student scans face to verify identity and mark presence. System checks Face, GPS, and Time.
Task Sequence	1. Student logs in to the app. 2. Navigates to "Mark Attendance" section. 3. Grants camera and location permissions. 4. Scans face for verification. 5. System validates Face + GPS + Time. 6. Student receives success notification.
Post-condition	Attendance recorded in database.

Table 1: Use Case Template: Student

Use Case Name	**Manual Override Attendance**
Actor	Teacher
Pre-condition	Teacher logged in; Student physically present but unable to scan.
Description	Teacher manually marks a student as present for a specific session due to technical issues.
Task Sequence	1. Teacher logs in. 2. Selects the active Class/Session. 3. Views the list of students. 4. Locates the specific student. 5. Clicks "Mark Present (Manual)". 6. Confirms the action.
Post-condition	Student marked present in database with "Manual" flag.

Table 2: Use Case Template: Teacher

Use Case Name	**Manage User Accounts**
Actor	Admin
Pre-condition	Admin logged in with secure credentials.
Description	Admin updates student or teacher details (e.g., Email, Phone) upon request.
Task Sequence	1. Admin logs in to the dashboard. 2. Navigates to "User Management". 3. Searches for the user by ID/Name. 4. Clicks "Edit Profile". 5. Updates contact details. 6. Saves changes.
Post-condition	User profile updated in Users Collection.

Table 3: Use Case Template: Admin

Use Case Name	**View Attendance Records**
Actor	All (Student, Teacher, Admin)
Pre-condition	User must be logged in to the system.
Description	Allows users to view attendance logs relevant to their role (Student: own logs; Teacher: class logs; Admin: all logs).
Task Sequence	1. User logs in to their respective dashboard. 2. Navigates to the "Attendance" or "Reports" section. 3. Selects relevant filters (Date Range, Course ID, or Student Name). 4. System retrieves data based on role permissions. 5. System displays the attendance data in tabular or graphical format.
Post-condition	Attendance data is displayed to the user.

Table 4: Use Case Template: View Attendance (All Actors)

2.2 Activity Diagram

The Activity Diagram illustrates the flow of control for marking attendance, including the decision nodes for Face Match, Geofence Verification, and Time validation.

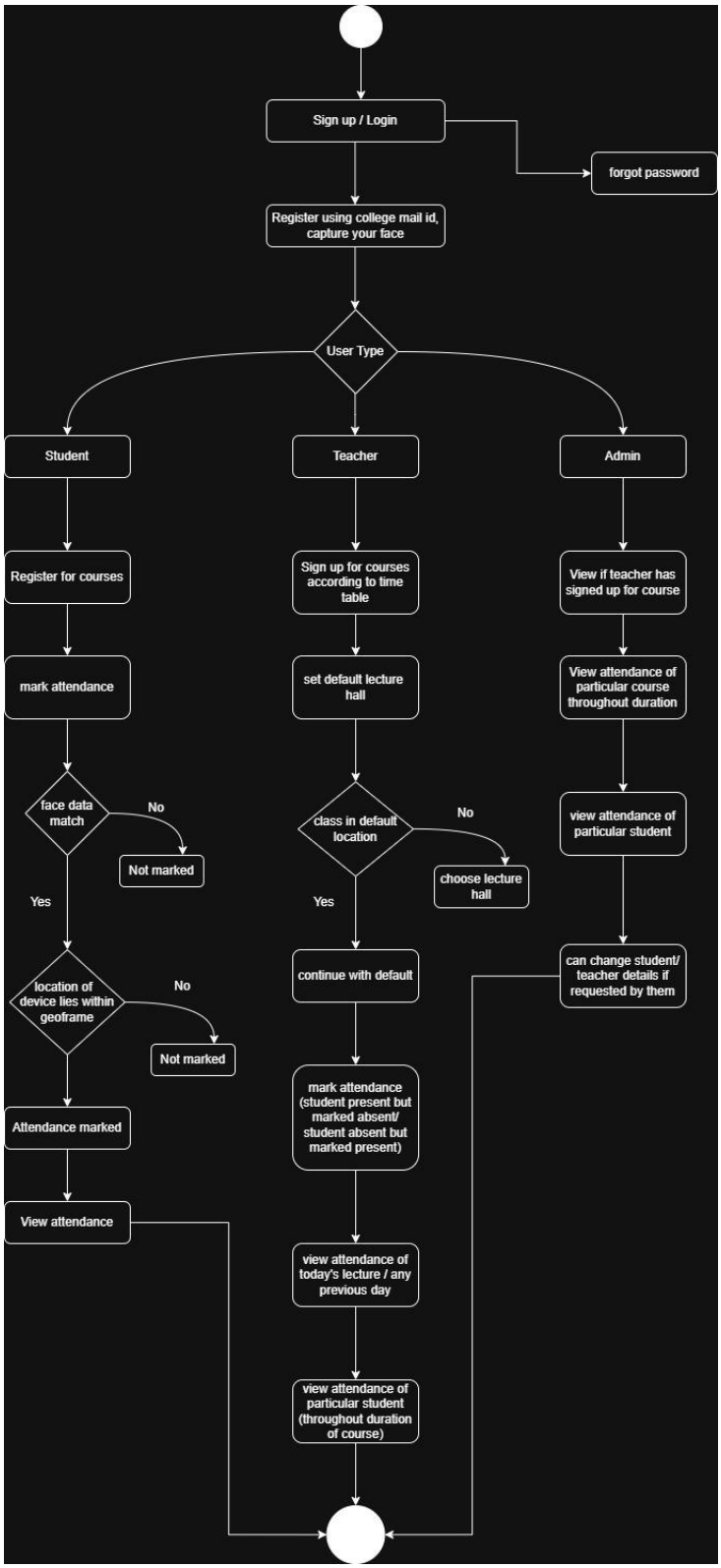

Figure 2: Activity Diagram (Attendance Logic)

2.3 Data Flow Diagrams (DFD)

2.3.1 DFD Level 0 (Context Diagram)

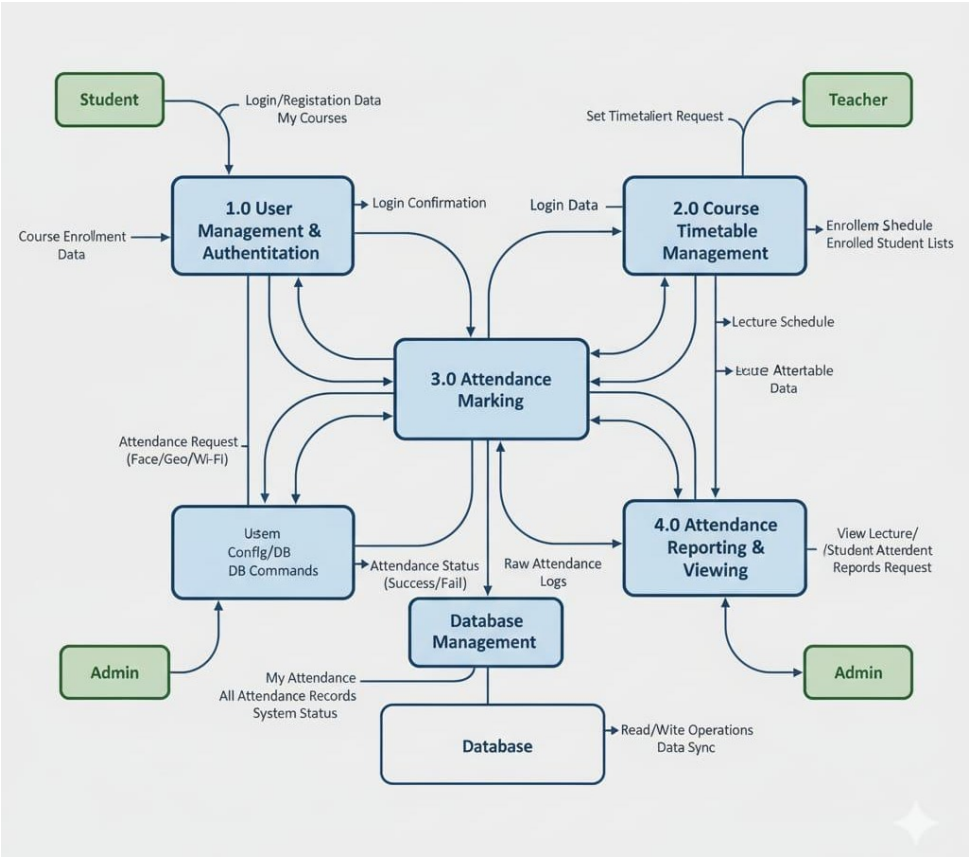

Figure 3: DFD Level 0: Context Diagram

2.3.2 DFD Level 1

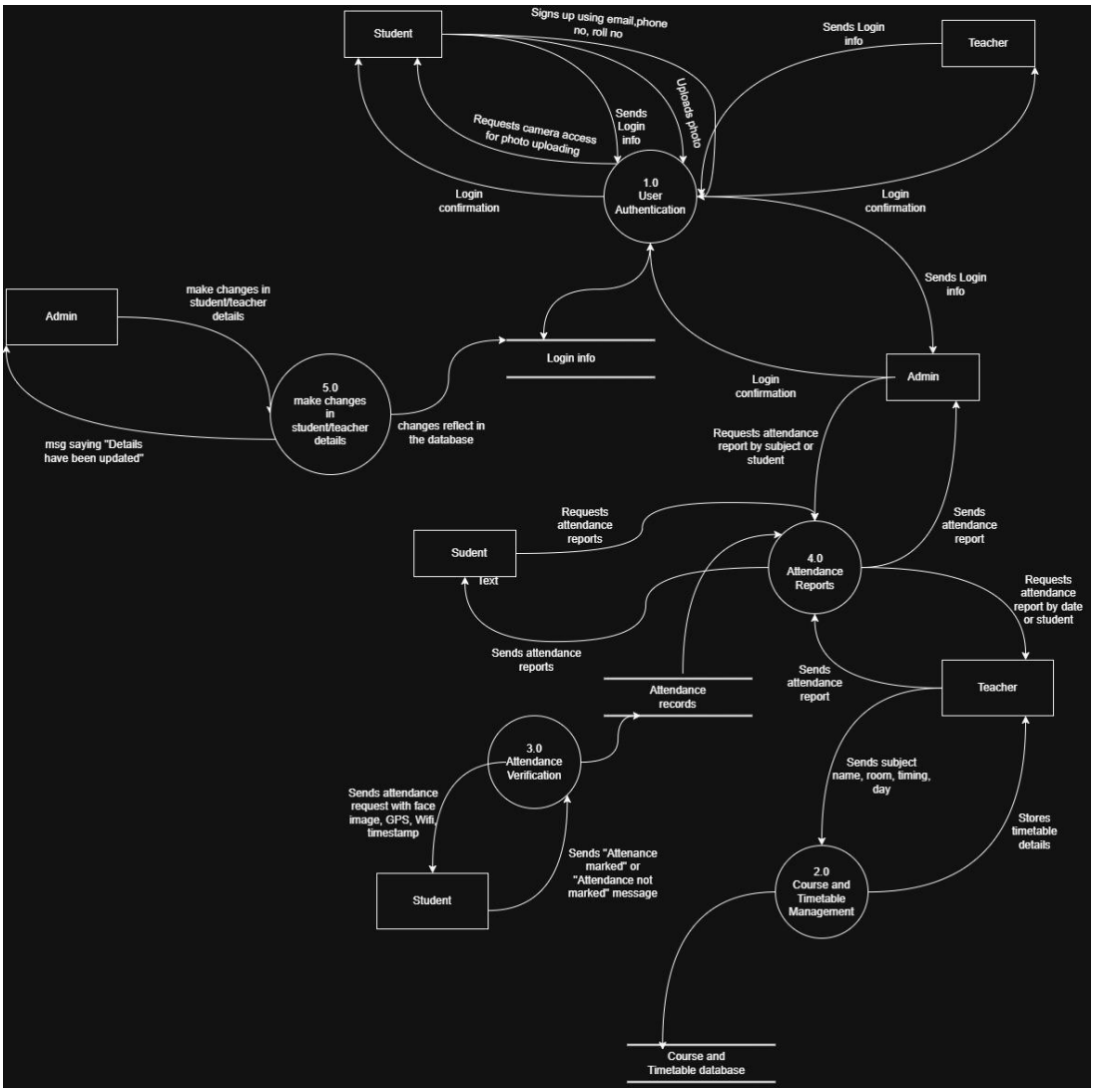

Figure 4: DFD Level 1: Detailed Data Flow

2.3.3 DFD Level 2

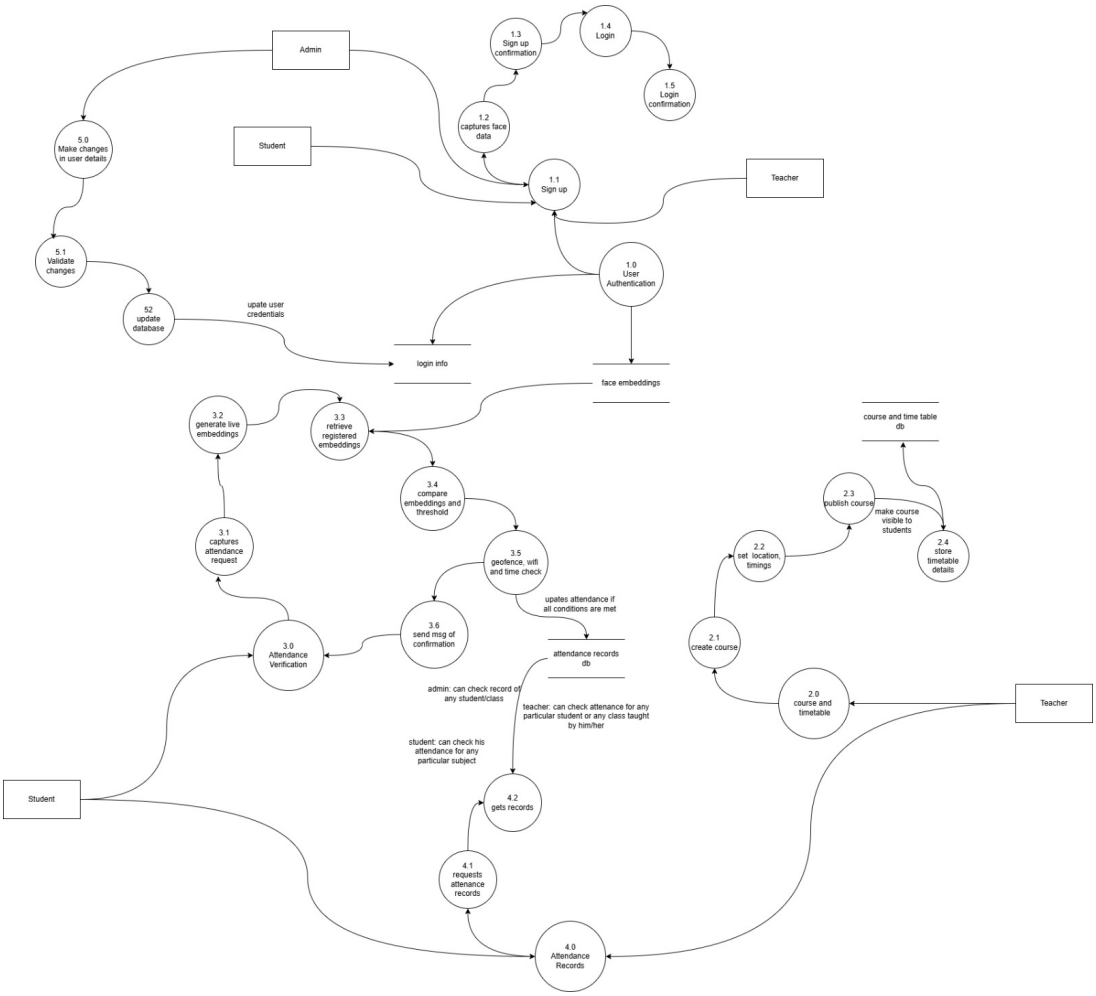

Figure 5: DFD Level 2: Component Detail

3 Software Requirements Specification (SRS)

Software Requirements Specification

For

AI-ENABLED SMART ATTENDANCE SYSTEM

Version 1.0

Prepared by:
Team: Tech Visionaries

Charu Goyal	102303113
Yuvraj Sharma	102303119
Nandini Puri	102303092
Paavni Sharma	102313052

Instructor: Dr. Vinod Kumar Bhalla
Course: Software Engineering
Subgroup: 3C14

3.1 Introduction

3.1.1 Purpose of this Document

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and detailed description of the **AI-ENABLED SMART ATTENDANCE SYSTEM**. This document describes the purpose, scope, interfaces, functional and non-functional requirements, software and hardware constraints, and design constraints of the system.

This SRS is intended for the stakeholders of the project, including the development team, project managers, and the end-users (students, teachers, and administrators), to ensure a shared understanding of the system to be developed. It will serve as the primary reference for the validation and verification of the final product.

3.1.2 Scope of the Development Project

The "AI-Enabled Smart Attendance System" is a web-based application designed to modernize the traditional attendance process in educational institutions. The system leverages advanced technologies such as Artificial Intelligence (specifically, Facial Recognition) and Geolocation services (GPS) to ensure accurate, secure, and efficient attendance marking.

The scope of the project includes the design, development, testing, and deployment of the following modules:

- **Student Module:** For registration, face enrollment, marking attendance via face scan, and viewing personal attendance history.
- **Teacher Module:** For course management, schedule creation, attendance monitoring, and manual overrides.
- **Admin Module:** For user management (students/teachers), system configuration, and generating institution-wide reports.
- **AI Engine:** For processing face embeddings using FaceNet and performing similarity checks.
- **Location Service:** For validating user coordinates against predefined classroom geofences.

3.1.3 Definitions, Abbreviations, and Acronyms

- **SRS:** Software Requirements Specification.
- **AI:** Artificial Intelligence.
- **FaceNet:** A deep learning model used for extracting high-quality features from face images (face embeddings).
- **Geofencing:** A virtual perimeter for a real-world geographic area.
- **GPS:** Global Positioning System.
- **API:** Application Programming Interface.

- **JWT:** JSON Web Token, used for secure transmission of information.
- **UI/UX:** User Interface / User Experience.
- **DBMS:** Database Management System.

3.1.4 References

1. *IEEE Std 830-1998*, IEEE Recommended Practice for Software Requirements Specifications.
2. *FaceNet: A Unified Embedding for Face Recognition and Clustering*, Schroff et al., Google.
3. *FastAPI Documentation*, https://fastapi.tiangolo.com/
4. *React.js Documentation*, https://react.dev/
5. *MongoDB Manual*, https://www.mongodb.com/docs/manual/

3.1.5 Overview

The remainder of this document is organized as follows:

- **Section 3.2 (Overall Description):** Describes the general factors that affect the product and its requirements, including user characteristics and constraints.
- **Section 3.3 (Specific Requirements):** Contains the detailed functional and non-functional requirements essential for the system's development.
- **Section 3.4 (System Models):** Provides data and logic models.

3.2 Overall Description

3.2.1 Product Perspective

The Smart Attendance System is a new, self-contained product. It is not a component of a larger system, although it is designed to potentially integrate with existing University ERP systems in the future via API endpoints. The system operates in a client-server architecture:

- **Client Side:** A Progressive Web App (PWA) built with React.js that runs on students' and teachers' mobile devices or laptops. It handles the UI, camera access, and GPS location retrieval.
- **Server Side:** A Python-based FastAPI server that handles business logic, AI processing (face matching), and database operations.
- **Database:** A MongoDB cloud instance that stores user profiles, face embeddings (vector data), course details, and attendance logs.

3.2.2 Product Functions

The major functions of the system are categorized by user role:

1. Student Functions

- **Registration & Login:** Secure authentication.
- **Face Enrollment:** One-time process to capture and store face data.
- **Check-in (Mark Attendance):** The core function where the student captures a live image. The system validates the face against the stored embedding and checks if the device is within the classroom's geofence.
- **View Dashboard:** View attendance percentage per subject and history logs.

2. Teacher Functions

- **Course Management:** Add or view assigned courses.
- **Session Creation:** Define class timings and physical location (latitude/longitude) for geofencing.
- **Live Monitoring:** View real-time attendance as students check in.
- **Manual Override:** Mark a student as present if they face technical difficulties.
- **Report Generation:** Export attendance sheets for a date range.

3. Admin Functions

- **User Management:** Create, update, or delete student/teacher accounts.
- **System Health:** Monitor API status and database connectivity.
- **Master Data Management:** Manage list of departments, courses, and classrooms.

3.2.3 User Characteristics

- **Students:** Technically literate, comfortable using smartphones and web apps. Expect a fast, responsive interface.
- **Teachers:** Varying technical skills. The interface for them must be simple, focused on tasks like "Start Class" and "View List".
- **Admins:** High technical literacy. require detailed logs and control over the system configuration.

3.2.4 General Constraints

- **Hardware:** The client device must have a working camera and GPS sensor.
- **Connectivity:** Stable internet connection is required for face verification and database updates.
- **Lighting:** Face recognition accuracy depends on lighting conditions during the scan.
- **Browser Support:** The system must support Chrome, Firefox, and Safari.

3.2.5 Assumptions and Dependencies

- It is assumed that all students possess a smartphone with a camera.
- It is assumed that GPS signals are available inside classrooms (or Wi-Fi geolocation fallback is used).
- The system depends on the availability of the MongoDB Atlas cloud service.

3.3 Specific Requirements

3.3.1 External Interface Requirements

1. User Interfaces

- **Login Screen:** Simple form for Email/Password with a "Forgot Password" link.
- **Student Dashboard:** specific cards for "Upcoming Classes", "Mark Attendance" button (active only during class time), and "Attendance Summary" charts.
- **Camera Interface:** A modal overlay accessing the device webcam with a visual guide (oval overlay) for face positioning.
- **Teacher Dashboard:** List of active courses, "Create Session" form with map integration for setting location, and a tabular view of student attendance.

2. Hardware Interfaces

- **Camera:** The application interacts with the device's camera via the browser's MediaDevices API (‘navigator.mediaDevices.getUserMedia‘). Minimum resolution required: 720p.
- **GPS Sensor:** The application accesses the Geolocation API (‘navigator.geolocation‘) to fetch Latitude and Longitude with high accuracy (‘enableHighAccuracy: true‘).

3. Software Interfaces

- **Operating System:** Platform independent (Web-based).
- **Database:** MongoDB for storing JSON-like documents.
- **AI Library:** TensorFlow/PyTorch for running the FaceNet model inference.

4. Communications Interfaces

- All client-server communication occurs over **HTTPS** using RESTful API endpoints.
- Data is exchanged in **JSON** format.

3.3.2 Functional Requirements

FR-01: Authentication Module

- **FR-01-01:** The system shall allow users to log in using University ID and Password.
- **FR-01-02:** The system shall lock the account after 5 failed login attempts.
- **FR-01-03:** The system shall maintain user sessions using JWT tokens.

FR-02: Face Recognition Module

- **FR-02-01 (Enrollment):** The system shall accept 3-5 images of the user during registration to generate a robust face embedding.
- **FR-02-02 (Verification):** During attendance, the system shall capture a live image and compare its embedding with the stored embedding using Cosine Similarity.

- **FR-02-03 (Threshold):** A match shall be confirmed only if the similarity score exceeds a threshold of 0.85 (configurable).

FR-03: Geofencing Module

- **FR-03-01:** The system shall allow teachers to set a "Classroom Location" by either selecting on a map or using their current device location.
- **FR-03-02:** The system shall define a valid radius (e.g., 10 meters) around the classroom center.
- **FR-03-03:** During attendance marking, the system shall compare the student's GPS coordinates with the classroom's coordinates.
- **FR-03-04:** If the distance > radius, attendance shall be rejected with the message "Location mismatch".

FR-04: Attendance Management

- **FR-04-01:** The system shall allow marking attendance only within the scheduled class time plus a buffer (e.g., 2 minutes).
- **FR-04-02:** The system shall prevent duplicate attendance entries for the same session.
- **FR-04-03:** The system shall log the timestamp, location, and confidence score for every attendance attempt.

3.3.3 Non-Functional Requirements

1. Performance Requirements

- **Latency:** The face verification process (image upload + processing + response) should take no longer than 10 seconds on a 4G network.
- **Throughput:** The backend shall handle at least 100 concurrent requests per second to support peak attendance times (start of classes).

2. Reliability and Availability

- **Availability:** The system shall be available 99.9% of the time during university working hours (8:00 AM to 6:00 PM).
- **Accuracy:** The False Acceptance Rate (FAR) for face recognition should be less than 0.1%.

3. Security Requirements

- **Data Privacy:** User face images should ideally not be stored raw; only vector embeddings should be stored. If images are stored, they must be encrypted.
- **Communication Security:** All data in transit must be encrypted using TLS 1.2 or higher.

- **Access Control:** Role-Based Access Control (RBAC) must be strictly enforced. Students cannot access Teacher APIs.

4. Maintainability

- The code shall be modular, following the MVC (Model-View-Controller) or similar pattern.
- The API documentation shall be automatically generated using Swagger/OpenAPI.

3.4 System Models

3.4.1 Data Logic

The system relies on a relational logic implemented in a NoSQL structure.

- **User Entity:** Contains ‘uid‘, ‘name‘, ‘role‘, ‘embedding_vector‘.
- **Course Entity:** Contains ‘course_id‘, ‘name‘, ‘teacher_id‘ (FK).
- **Session Entity:** Contains ‘session_id‘, ‘course_id‘ (FK), ‘start_time‘, ‘end_time‘, ‘geo_lat‘, ‘geo_long‘, ‘radius‘.
- **Attendance Entity:** Contains ‘record_id‘, ‘session_id‘ (FK), ‘student_id‘ (FK), ‘timestamp‘, ‘status‘.

3.5 Change History

Version	Date	Key Changes & Features
v1.5.0	2025-12-05	**Stability & Optimization** - Optimization: Reduced face verification latency by 30%. - Fix: Resolved GPS drift issue on Android devices. - UI: Polished dashboard animations and responsive layouts.
v1.4.0	2025-11-28	**Geofencing & Real-time Features** - Geofencing: Implemented dynamic radius adjustment for classrooms. - Real-time: Added live socket updates for teacher’s attendance view. - Notifications: Integrated push notifications for successful check-ins.
v1.3.0	2025-11-15	**Security & Reporting** - Security: Implemented Argon2 hashing for passwords. - Reporting: Added ”Export to PDF” feature for attendance sheets. - Admin: Added system health monitoring dashboard.

v1.2.0	2025-10-20	**Teacher Module & Manual Override** - Teacher: Added "Create Session" and "Manual Override" functionality. - UI: Improved mobile view for teacher dashboard. - Bug Fix: Resolved login token expiration issues.
v1.1.0	2025-09-10	**Face Recognition Integration** - Core: Integrated FaceNet model for embedding generation. - Student: Added "Face Enrollment" flow with 3-image capture. - Database: Optimized vector storage in MongoDB.
v1.0.0	2025-08-25	**Initial Release & PWA Setup** - Foundation: Initial project setup with React and FastAPI. - PWA: Configured manifest and service workers for offline capability. - Auth: Basic Login/Signup with JWT authentication.

3.6 Document Approvers

Name	**Role**	**Signature**
Dr. Vinod Kumar Bhalla	Project Mentor	

4 Design Phase

4.1 Class Diagram

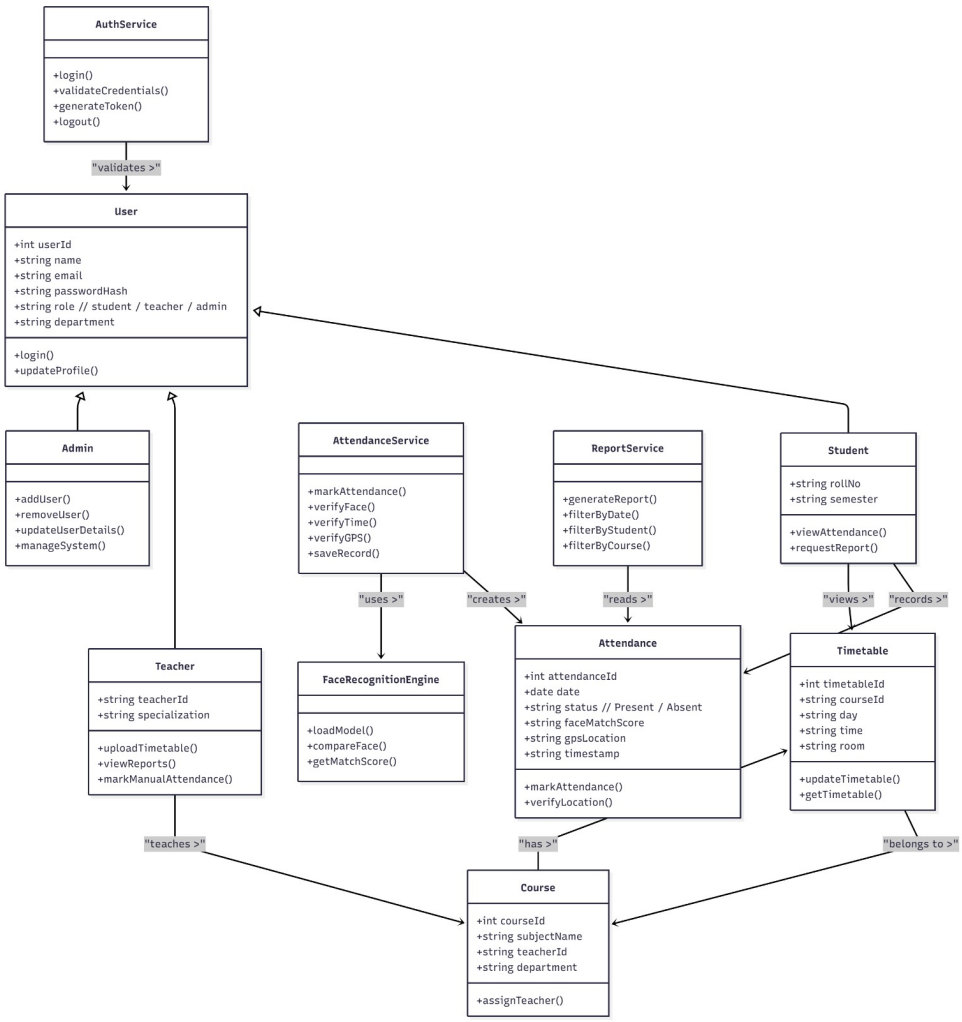

Figure 6: Class Diagram

4.1.1 Significant Cases

Significant Case 1: Automated Face Verification
Classes Involved: `User`, `FaceEmbedding`, `AttendanceService`, `Session`
Relationship: One-to-Many (One User has one Embedding; One Session has many Verification Requests).
Explanation: This logic forms the core security layer of the system to prevent proxy attendance:

1. The `User` entity holds a stored 'embedding_vector' generated during the registration phase.
2. When a student initiates a check-in, the `AttendanceService` receives a live image capture.
3. The AI Engine processes the live image to extract a temporary feature vector.
4. The system calculates the **Cosine Similarity** between the stored vector and the live vector.
5. If the similarity score exceeds the defined threshold (e.g., 0.85), the identity is verified, and the process moves to location checking.

Significant Case 2: Geofence Location Validation
Classes Involved: `Session`, `LocationService`, `Student`, `AttendanceRecord`
Relationship: Many-to-One (Many Students attempt validation against One Session's location).
Explanation: This case ensures physical presence in the classroom:

1. A `Session` object is initialized by the Teacher with specific 'geo_lat', 'geo_long', and a valid 'radius' (e.g., 50 meters).
2. The Student's client application retrieves high-accuracy GPS coordinates via the Geolocation API.
3. The `LocationService` computes the distance between the Student's coordinates and the Session's center using the **Haversine Formula**.
4. If the calculated distance is less than or equal to the session radius, the location check passes.
5. Upon successful Face and Location verification, an `AttendanceRecord` is created and persisted in the database.

4.2 Sequence Diagram

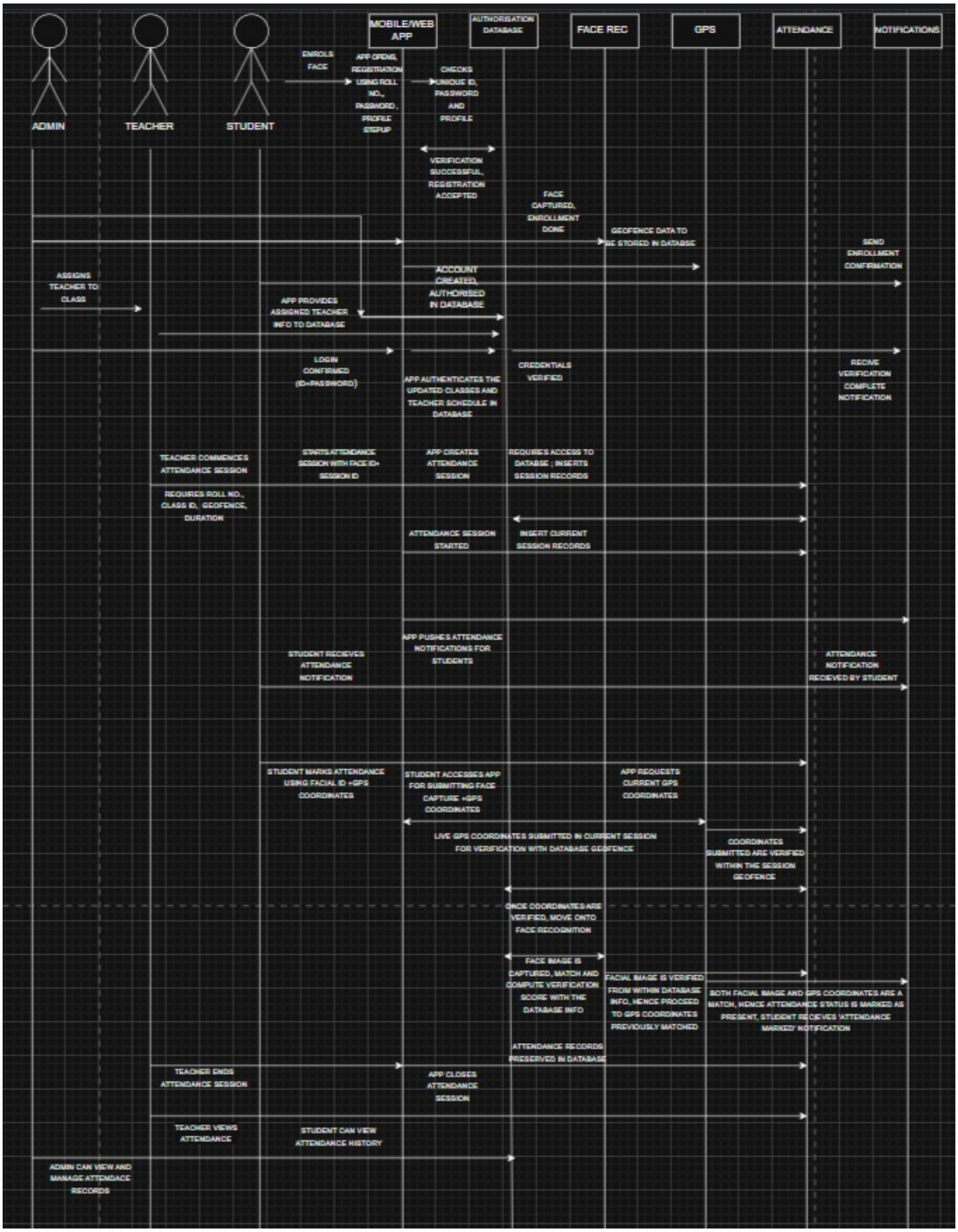

Figure 7: Sequence Diagram

4.3 Collaboration Diagram

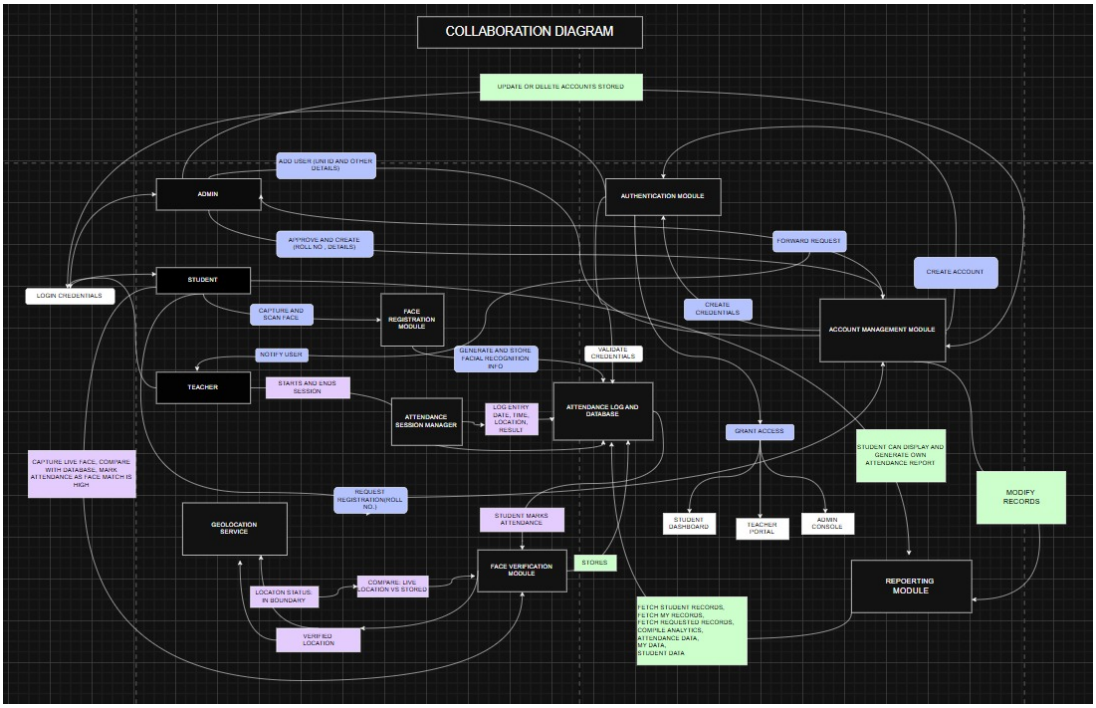

Figure 8: Collaboration Diagram

Significant Case 3: Attendance Correction Request Lifecycle
Context: This case details the state transitions of an attendance correction request initiated by a student when automated marking fails (e.g., due to low lighting or GPS drift).
Entities: `Student` (Requester), `Teacher` (Approver), `AttendanceRequest`
Flow:

1. **Request Initiation:** A Student views an 'Absent' status for a class they physically attended. They click "Request Correction". An `AttendanceRequest` object is created with state **PENDING**.

2. **Notification:** The system triggers a real-time notification to the Teacher dashboard for the specific course.

3. **Decision - Approve:** If the Teacher validates the student's presence (e.g., via visual check or memory), they click "Approve".

   - The request state transitions to **APPROVED**.
   - The system creates or updates the `AttendanceRecord` to 'Present' with a 'Manual' flag.

4. **Decision - Reject:** If the Teacher denies the claim, they click "Reject". The request state transitions to **REJECTED**. No changes occur in the attendance log.

5. **Cancellation:** If the Student withdraws the request before the Teacher acts, the state becomes **CANCELLED** (Soft Delete).

5 Deployment Phase

5.1 Deployment Diagram

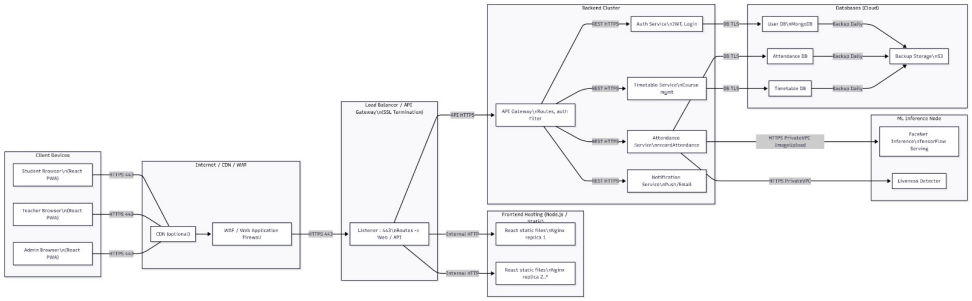

Figure 9: Deployment Diagram for AI-ENABLED SMART ATTENDANCE SYSTEM

Deployment Nodes Description:

- **Client Tier:** User's device (Mobile/Desktop) running the PWA via a web browser. Handles UI rendering, camera access for face scanning, and local state management.
- **Application Server (Render/Cloud):**
  - **Frontend Service:** Node.js environment serving the React build artifacts.
  - **Backend Service:** Python environment running the FastAPI server with Uvicorn workers to handle AI inference and logic.
- **Data Tier:** Persistent storage managed via MongoDB Atlas (Cloud), storing user profiles, face embeddings (vector data), and attendance logs.
- **External Services:** Google Maps API for location services and geofencing validation.

5.2 Showcase of Implemented Version

This section demonstrates the final deployed version of the "AI-ENABLED SMART ATTENDANCE SYSTEM," highlighting the core features accessible to end-users.

1. Login Interface

The secure entry point for Students, Teachers, and Admins to access the system.

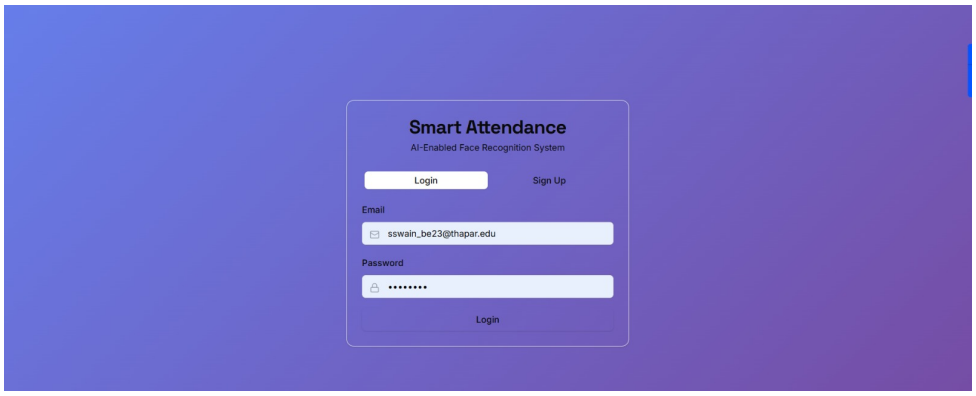

Figure 10: Implemented Login Interface

2. Student Dashboard

The main landing interface where students can view their attendance summary and initiate a check-in.

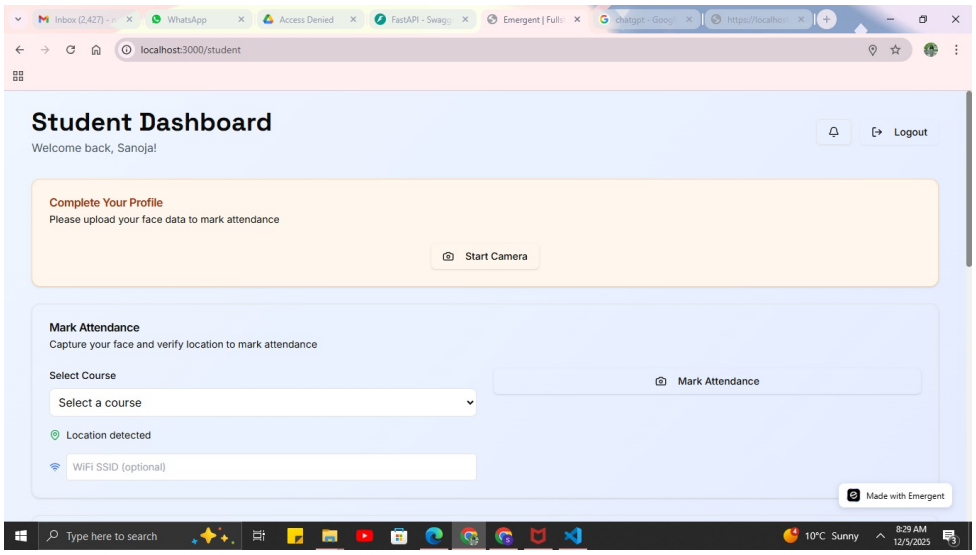

Figure 11: Implemented Student Dashboard